Solution: Minimum Difference Between BST Nodes (easy)

Problem Statement

Given a Binary Search Tree (BST), you are required to find the smallest difference between the values of any two different nodes.

In a BST, the nodes are arranged in a manner where the value of nodes on the left is less than or equal to the root, and the value of nodes on the right is greater than the root.

Example

Example 1:

- Input:

```
    4
   / \
  2   6
 / \
1   3
```

- Expected Output: 1
- Justification: The pairs (1,2), (2,3), or (3,4) have the smallest difference of 1.

Example 2:

- Input:

```
    10
   /  \
  5   15
 / \    \
2   7    18
```

- Expected Output: 2
- Justification: The pair (5,7) has the smallest difference of 2.

Example 3:

- Input:

```
  40
   \
    70
   /  \
  50  90
```

- Expected Output: 10
- Justification: The pair (40,50) has the smallest difference of 10.

Solution

A binary search tree has an interesting property where if you perform an in-order traversal, the result is a list of numbers in ascending order.

To solve this problem, you'll first perform an in-order traversal of the Binary Search Tree (BST) and store elements in the array. Once the traversal is complete and the list is fully populated, the next step is to find the minimum difference between adjacent elements in this list. Since the list is sorted, the smallest difference between any two nodes in the BST will be among these adjacent pairs.

1. **In-Order Traversal**: Start with the root of the tree and traverse the tree in in-order (left, root, right). This will give us the nodes in ascending order.
2. **Calculate Minimum Difference**: Once the in-order traversal is done and we have the values in ascending order, iterate through the list and calculate the difference between each pair of adjacent nodes.
3. **Return the Minimum**: Return the smallest difference.

Algorithm Walkthrough:

For the tree:

```
    4
   / \
  2   6
 / \
1   3
```

- Perform in-order traversal: Resulting list is [1, 2, 3, 4, 6]
- Calculate the differences: 1 (between 1 & 2), 1 (between 2 & 3), 1 (between 3 & 4), and 2 (between 4 & 6).
- The smallest difference is 1.

Here is the visual representation of the algorithm:

We will perform a inorder traversal(Left, Root, Right) and hence store the sorted values in a list. On that list, we will calculate the difference between two adjacent elements and hence a min difference will be returned

Since, NULL is passed and exit condition will be called and hence we will traverse back to node 1 and 1 will be added to the nodes list

Since, NULL is passed and exit condition will be called and hence we will traverse back to node 2

Now 2 will be added in nodes list

Now, we will move to 3 and then 3's left. Hence exit condition will be triggered and we will move to 3 again

Now, exit condition will be called and we will move to node 4

Now, mode 4 will be added in the list

Now, we will move to 6 with the same steps that we have followed earlier

Hence, the nodes list becomes:

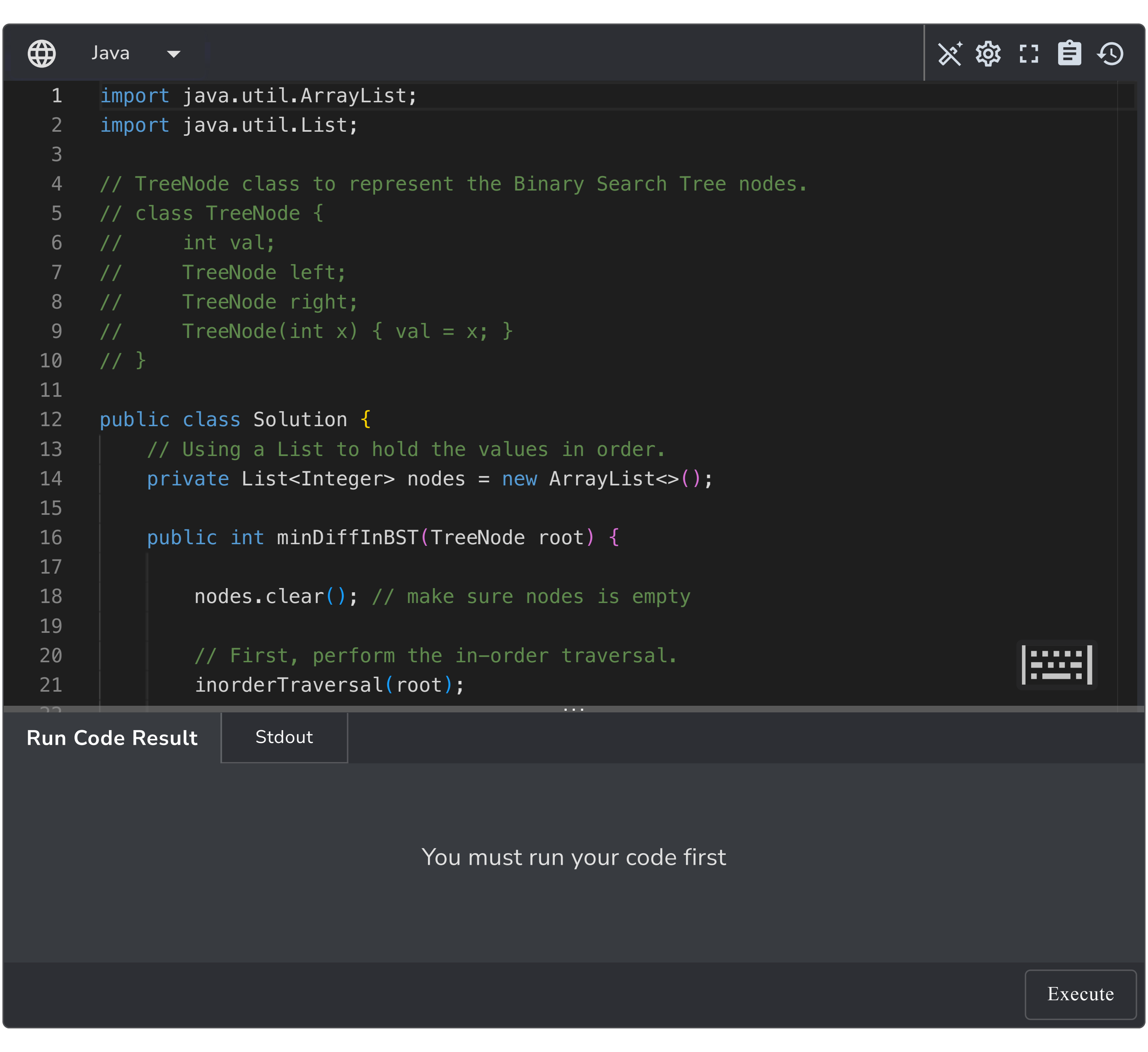
We will start the loop from index 1 and hence will get a +ve difference because the elements are sorted in ascending order

Since min. difference is 1 so 1 will be returned

Min Diff between BST nodes

Code

Here is the code for this algorithm:



```java
import java.util.ArrayList;
import java.util.List;

// TreeNode class to represent the Binary Search Tree nodes.
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

public class Solution {
    // Using a List to hold the values in order.
    private List<Integer> nodes = new ArrayList<>();

    public int minDiffInBST(TreeNode root) {

        nodes.clear(); // make sure nodes is empty

        // First, perform the in-order traversal.
        inorderTraversal(root);
```

You must run your code first

Complexity Analysis

- *Time Complexity:* `O(N)` - We traverse each node once during the in-order traversal, where N is the number of nodes.
- *Space Complexity:* `O(N)` - We store the values of all nodes in a list.

← Previous
Minimum Difference Between BST Nodes (Easy)
Next →
Range Sum Of BST (Easy)
☐ Mark as Completed